

Lab 10: Minimalizacja funkcji 1-D

Vladyslav Humeniuk

1 Introduction

Celem niniejszego laboratorium było zapoznanie się z algorytmami *one-dimensional minimization* dostarczonymi przez bibliotekę GNU Scientific Library (GSL) [1]. Problemy typu *optimization* polegają na znalezieniu minimum x^* takiego, że $f(x^*) \leq f(x)$ dla wszystkich x w danym zbiorze [2]. Algorytmy minimalizacji GSL rozpoczynają poszukiwania od ograniczonego obszaru, o którym wiadomo, że zawiera minimum, opisanego przez *lower bound* a oraz *upper bound* b , wraz z początkowym oszacowaniem m dla lokalizacji minimum [1]. Niniejsze sprawozdanie bada *computational complexity* oraz wymagania dotyczące *bracketing* dla trzech różnych iteracyjnych *minimizers*: metody Brenta, zmodyfikowanej metody Brenta (`quad_golden`) oraz *Golden Section search*.

2 Methodology

Zadania obliczeniowe zostały podzielone na następujące etapy:

1. **Analiza `brent_minim.c`:** Zbadanie bazowego programu, który wykorzystuje algorytm Brenta do znalezienia minimum funkcji $f(x) = \cos(x) + 1$ w przedziale $[0, 6]$ z początkowym przybliżeniem (guess) wynoszącym 2.0 [1].
2. **Porównanie algorytmów:** Modyfikacja programu bazowego w celu uwzględnienia metod `gsl_min_fminimizer_goldensection` oraz `gsl_min_fminimizer_quad_golden` i porównanie ich tempa zbieżności (*convergence rates* w postaci liczby iteracji).
3. **Problematic Bracketing:** Zastosowanie algorytmów do nowej funkcji, $f(x) = e^{-x} - \sin(x) + \sqrt{x}$, w przedziale $[\pi/2, 4\pi]$, w celu analizy przyczyn błędów (*failure modes*) i implementacji skorygowanej strategii dla *bracketing*.

3 Results

3.1 Task 1 & 2: Optimization of $f(x) = \cos(x) + 1$

Początkowy program wykorzystuje algorytm Brenta do znalezienia minimum funkcji $f(x) = \cos(x) + 1$, które teoretycznie znajduje się w punkcie $x = \pi$ [1]. Wyjściowy przedział to $(0, 6)$, z początkowym przybliżeniem minimum na poziomie 2 [1]. Wykres tej funkcji przedstawiono na Rysunku 1.

Wyniki zbieżności (*convergence*) dla trzech testowanych metod przedstawiono w Tabeli 1.

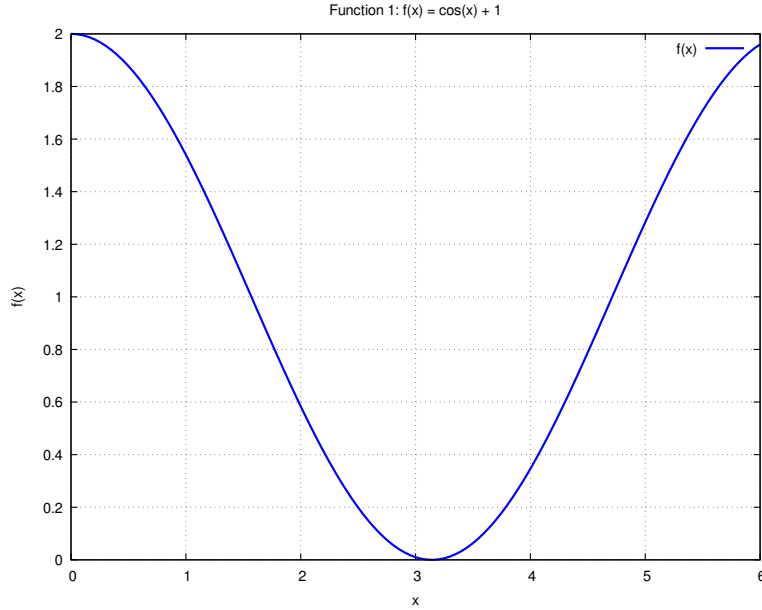


Figure 1: Wykres funkcji $f(x) = \cos(x) + 1$. Globalne minimum jest wyraźnie zlokalizowane w punkcie $x \approx 3.14$.

Table 1: Porównanie wydajności algorytmów minimalizacji 1D dla $f(x) = \cos(x) + 1$.

Method	Iterations	Found Minimum (x)	$f(x)$
Brent	6	3.14159	0.00000
Quad-Golden	19	3.14165	0.00000
Golden Section	24	3.14157	0.00000

Porównanie złożoności obliczeniowej (*Computational Complexity*):

Metoda Brenta jest wysoce wydajna, wymagając zaledwie 6 iteracji do zbieżności. Algorytm osiąga tę prędkość poprzez aproksymację funkcji za pomocą paraboli interpolacyjnej (*interpolating parabola*) przechodzącej przez trzy znane punkty, co pozwala mu na bezpośredni przeskok w stronę minimum [1]. *Golden Section search* jest z kolei metodą najwolniejszą (24 iteracje), ponieważ opiera się na ściślejszej geometrycznej redukcji przedziału, nie czyniąc przy tym żadnych założeń dotyczących kształtu funkcji. Wariant `quad_golden`, wykorzystujący algorytm *safeguarded step-length* autorstwa Gilla i Murraya, potrzebował 19 iteracji [1].

3.2 Task 3: Optimization of $f(x) = e^{-x} - \sin(x) + \sqrt{x}$

Druga funkcja, $f(x) = e^{-x} - \sin(x) + \sqrt{x}$, została poddana analizie w wymaganym przedziale $[\pi/2, 4\pi]$. Jej zachowanie zilustrowano na Rysunku 2.

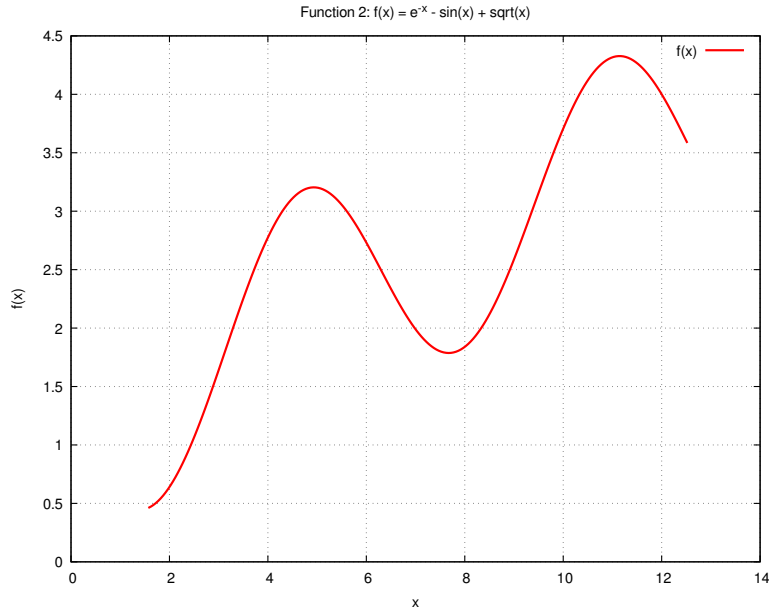


Figure 2: Wykres funkcji $f(x) = e^{-x} - \sin(x) + \sqrt{x}$. Lokalne minimum jest widoczne w pobliżu $x = 7.67$, ale absolutnie najniższa wartość w przedziale $[\pi/2, 4\pi]$ znajduje się na lewym brzegu.

Analiza problemu *Bracketing*: Podczas próby inicjalizacji *minimizers* biblioteki GSL na przedziale $[\pi/2, 4\pi]$ z przybliżeniem (guess) wynoszącym $x = 4.0$, program zwraca błąd krytyczny: `invalid argument supplied by user`.

Sytuacja ta ma miejsce, ponieważ GSL wymaga ścisłego spełnienia warunków *bracketing*. Warunkiem matematycznym jest to, aby wartość funkcji w przybliżeniu wewnętrznym m była bezwzględnie mniejsza niż jej wartości na obu krańcach przedziału: $f(m) < f(a)$ oraz $f(m) < f(b)$. Jednakże w przedziale $[\pi/2, 4\pi]$, absolutne minimum przyjmuje wartość na lewym brzegu $a = \pi/2$ (gdzie $f(a) \approx 0.461194$). Ponieważ wartość na brzegu jest niższa od jakiegokolwiek wewnętrznego minimum lokalnego, podanie poprawnej wartości m spełniającej restrykcyjne warunki *bracketing* jest niemożliwe.

Rozwiązanie i skorygowane wyniki: Aby skutecznie zlokalizować minimum lokalne, początkowy *bracket* (przedział poszukiwań) musi zostać zwężony w taki sposób, aby obejmował wyłącznie “dołek” (*dip*) krzywej, całkowicie omijając problematyczny lewy brzeg. Przedział został poprawiony na $[6.0, 9.0]$, ze zmienionym przybliżeniem początkowym 7.0.

Wyniki wydajności algorytmów na tak skorygowanym przedziale zestawiono w Tabeli 2.

Table 2: Porównanie wydajności algorytmów minimalizacji 1D dla $f(x) = e^{-x} - \sin(x) + \sqrt{x}$ w poprawionym przedziale $[6.0, 9.0]$.

Method	Iterations	Found Minimum (x)	$f(x)$
Brent	6	7.67295	1.78682
Quad-Golden	17	7.67292	1.78682
Golden Section	21	7.67296	1.78682

Gdy tylko poprawne granice *bracket* zostały ustanowione, hierarchia wydajności poszczególnych algorytmów pozostała identyczna jak w pierwszym zadaniu. Metoda Brenta osiągnęła zbieżność wyjątkowo szybko (w 6 iteracjach), podczas gdy *Golden Section* wymagał ponad trzykrotnie większej liczby kroków (21 iteracji).

4 Conclusion

Zadania laboratoryjne skutecznie zobrazowały działanie algorytmów *one-dimensional optimization*. Metoda Brenta dowiodła, że jest podejściem najbardziej zoptymalizowanym obliczeniowo (*computationally efficient*), wykorzystującym *parabolic interpolation* w celu uzyskania szybkiej zbieżności. Przeprowadzone eksperymenty uwypukliły również kluczowe ograniczenie *minimizers* opartych na koncepcji przedziałów (*bracket-based*): nie są one w stanie zlokalizować minimów globalnych leżących bezpośrednio na krańcach przedziału, o ile nie istnieje w jego wnętrzu żaden niższy punkt. Poprawna wizualizacja problemu i ręczne dostrajanie granic (*manual bounding*) są koniecznym warunkiem dla zautomatyzowanych procedur klasy *optimization*.

Bibliography

- [1] Free Software Foundation: GNU Scientific Library Reference Manual: One Dimensional Minimization (2009), <https://www.gnu.org/software/gsl/doc/html/min.html>, accessed strictly for GSL minimization routines
- [2] Heath, M.T.: Scientific computing: An introductory survey, chapter 6 - optimization. Department of Computer Science, University of Illinois at Urbana-Champaign (2002), http://artemis.wszib.edu.pl/~funika/mois/lab10/heath_optim.pdf